EXPERIMENT-3

SACHIN SINGH

24UADS1045

Objective :-

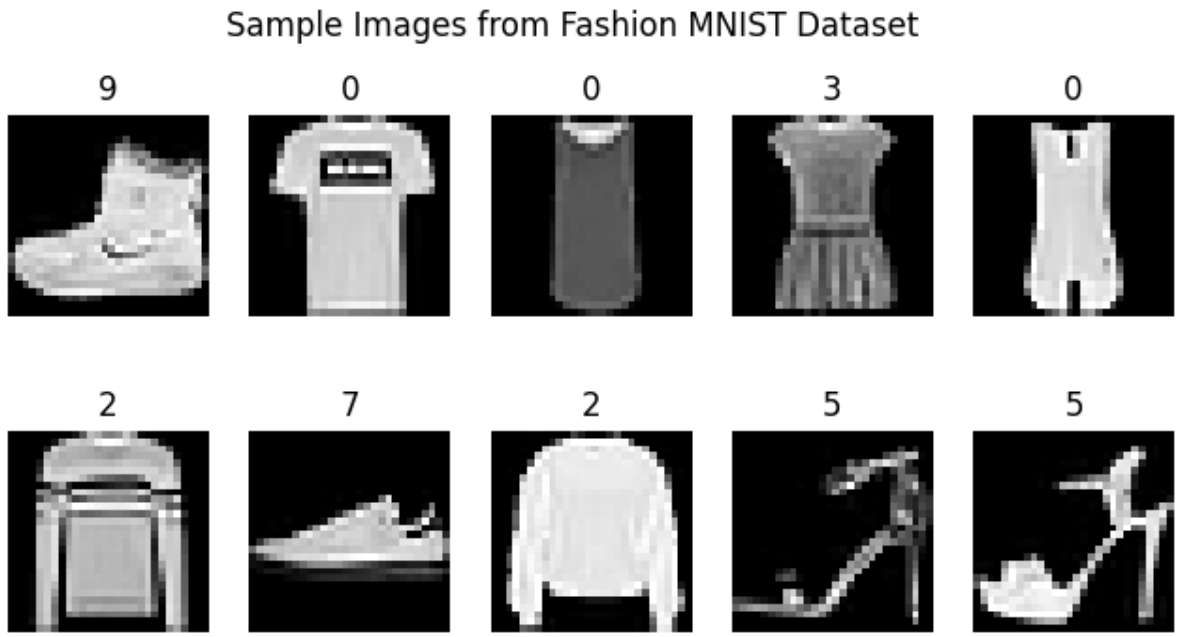
WAP to train and evaluate a convolutional neural network using Keras Library to classify MNIST fashion dataset. Demonstrate the effect of filter size, regularization, batch size and optimization algorithm on model performance.

Description Of the Model :-

Introduction to the Model:-
The model used in this experiment is a Convolutional Neural Network (CNN) developed to classify images from the Fashion-MNIST dataset. The dataset contains grayscale images of size 28 × 28 pixels representing different clothing categories such as shirts, trousers, bags, shoes, and coats. The CNN automatically learns visual patterns from the images and classifies them into one of the ten classes.

Description of Dataset :-

The dataset used in this experiment is the **Fashion-MNIST dataset**, which is a widely used benchmark dataset for image classification tasks in deep learning. It was created as a replacement for the traditional MNIST digit dataset and contains images of various fashion products. The dataset is provided by Zalando Research and is commonly used to evaluate machine learning and deep learning algorithms.



Convolution Layers for Feature Extraction:-
Convolution layers are responsible for extracting meaningful features from the input images. These layers apply learnable filters that move across the image and detect important patterns such as edges, shapes, and textures.

- Convolution filters slide across the image to capture spatial features.
- Each filter generates a feature map highlighting specific patterns.
- The network learns the optimal filter weights during training through backpropagation.

Pooling Layers for Dimensionality Reduction:-
Pooling layers are applied after convolution layers to reduce the spatial dimensions of the feature maps. This reduces computational complexity and helps the model focus on the most important features.

- Max pooling selects the maximum value from each region of the feature map.
- It reduces the size of feature maps and improves computational efficiency.
- Pooling helps make the model robust to small image variations.

Flatten Layer for Data Transformation:-
The flatten layer converts the two-dimensional feature maps into a one-dimensional vector so that it can be processed by fully connected layers.

- Transforms feature maps into a vector format.
- Prepares extracted features for classification in dense layers.

Fully Connected Dense Layers :-
Dense layers perform the final classification based on the extracted features.

- These layers learn complex relationships between image features and class labels.
- Weights in these layers are updated during training to improve prediction accuracy.

Output Layer and Softmax Activation:-
The final output layer uses the softmax activation function to produce probability values for each class.

- Softmax converts output scores into probability values.
- The class with the highest probability is selected as the predicted label.

Loss Function and Optimization:-
The model is trained using the categorical cross-entropy loss function and the Adam optimizer.

- The loss function measures the difference between predicted and actual labels.
- Adam optimizer updates model weights efficiently using adaptive learning rates.
- This helps the model converge faster during training.

Training Process and Model Learning:-
The model is trained for several epochs using batches of training data. During training, the backpropagation algorithm adjusts the weights of the network to minimize loss and improve classification accuracy.

- Each epoch allows the model to learn from the entire training dataset.
- Batch training improves computational efficiency and stability.
- Accuracy increases and loss decreases as training progresses.

Performance Analysis of the Model :-
Different graphs are used to analyze the performance of the model and understand the impact of hyperparameters.

- Training accuracy vs epoch graph shows learning progress.
- Training loss vs epoch graph shows how error decreases during training.
- Filter size vs accuracy graph analyzes feature extraction capability.
- Batch size vs accuracy graph studies training efficiency.
- Optimizer comparison graph evaluates optimization performance.
- Regularization comparison graph analyzes overfitting control.

Overall Model Performance :-
The CNN model effectively learns hierarchical image features and performs accurate classification on the Fashion-MNIST dataset.

- CNN automatically extracts spatial features from images.
- The model achieves high classification accuracy with efficient computation.
- Hyperparameter tuning improves the overall performance and generalization ability of the network.

Source Code : -

```
import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

import numpy as np

import matplotlib.pyplot as plt


(x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()

x_train = x_train/255.0

x_test = x_test/255.0

x_train = x_train.reshape(-1,28,28,1)

x_test = x_test.reshape(-1,28,28,1)


plt.figure(figsize=(8,4))

for i in range(10):

    plt.subplot(2,5,i+1)

    plt.imshow(x_train[i].reshape(28,28), cmap='gray')

    plt.title(y_train[i])
```

```
    plt.axis('off')
plt.show()

def create_model(filter_size=3, optimizer='adam', reg=None):

    model = keras.Sequential([
        layers.Conv2D(32,(filter_size,filter_size),activation='relu',
                      kernel_regularizer=reg,input_shape=(28,28,1)),
        layers.MaxPooling2D((2,2)),

        layers.Conv2D(64,(filter_size,filter_size),activation='relu',
                      kernel_regularizer=reg),
        layers.MaxPooling2D((2,2)),

        layers.Flatten(),

        layers.Dense(128,activation='relu',
                     kernel_regularizer=reg),

        layers.Dense(10,activation='softmax')
    ])

    model.compile(
        optimizer=optimizer,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

class EpochDetails(keras.callbacks.Callback):
```

```
    def on_epoch_end(self, epoch, logs=None):

        print("\nEpoch:",epoch+1)

        print("Loss:",logs['loss'])
        print("Accuracy:",logs['accuracy'])

        print("\nLayer Weights Summary:")

        for layer in self.model.layers:

            weights = layer.get_weights()

            if len(weights) > 0:

                w = weights[0]

                print(layer.name,
                      "mean weight:",np.mean(w),
                      "std:",np.std(w))

model = create_model(filter_size=3,
                     optimizer='adam',
                     reg=None)

history = model.fit(
    x_train,y_train,
    epochs=10,
    batch_size=128,
    validation_data=(x_test,y_test),
```

```
    callbacks=[EpochDetails()]
)

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(['train','test'])
plt.show()

plt.figure()

plt.plot(history.history['val_accuracy'])

plt.title("Validation Accuracy vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.show()

test_loss,test_acc = model.evaluate(x_test,y_test)

print("Test Accuracy:",test_acc)

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title("Model Loss")
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
plt.legend(['train','test'])
plt.show()

plt.figure()

plt.plot(history.history['val_loss'])

plt.title("Validation Loss vs Epoch")
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.show()

filters = [3,5,7]
results = []
accuracy = results


for f in filters:

    model = create_model(filter_size=f)

    h = model.fit(x_train,y_train,
                  epochs=5,
                  batch_size=128,
                  verbose=0)

    results.append(h.history['accuracy'][-1])

plt.bar(filters,results)
```

```
plt.xlabel("Filter Size")
plt.ylabel("Accuracy")
plt.title("Effect of Filter Size")
plt.show()

plt.plot(filters, accuracy, marker='o')

plt.xlabel("Filter Size")
plt.ylabel("Accuracy")
plt.title("Filter Size vs Accuracy Trend")

plt.show()

batch_sizes = [32,64,128]
results = []

for b in batch_sizes:

    model = create_model()

    h = model.fit(x_train,y_train,
                  epochs=5,
                  batch_size=b,
                  verbose=0)

    results.append(h.history['accuracy'][-1])

plt.plot(batch_sizes,results)

plt.xlabel("Batch Size")
```

```
plt.ylabel("Accuracy")
plt.title("Effect of Batch Size")
plt.show()

optimizers = ['adam','sgd','rmsprop']
results = []

for opt in optimizers:

    model = create_model(optimizer=opt)

    h = model.fit(x_train,y_train,
                  epochs=5,
                  batch_size=128,
                  verbose=0)

    results.append(h.history['accuracy'][-1])

plt.bar(optimizers,results)

plt.title("Optimizer Comparison")
plt.ylabel("Accuracy")
plt.show()

from tensorflow.keras import regularizers

regs = [None,
        regularizers.l2(0.001)]

labels = ['No Regularization','L2']
```

```
results = []

for r in regs:

    model = create_model(reg=r)

    h = model.fit(x_train,y_train,
                  epochs=5,
                  batch_size=128,
                  verbose=0)

    results.append(h.history['accuracy'][-1])

plt.bar(labels,results)

plt.title("Regularization Effect")
plt.ylabel("Accuracy")
plt.show()
```

Output :-

Epoch 1/10

469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 8ms/step - accuracy: 0.7200 - loss: 0.7845

Epoch: 1

Loss: 0.5448653697967529

Accuracy: 0.803683340549469

Layer Weights Summary:

conv2d mean weight: 0.015195904 std: 0.11303154

conv2d_1 mean weight: 0.0044258526 std: 0.06372525

dense mean weight: 0.0008895673 std: 0.04080557

dense_1 mean weight: -0.012676743 std: 0.13210112

469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **9s** 11ms/step - accuracy: 0.7202 - loss: 0.7840 - val_accuracy: 0.8580 - val_loss: 0.3939

Epoch 2/10

462/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step - accuracy: 0.8707 - loss: 0.3626

Epoch: 2

Loss: 0.3501933515071869

Accuracy: 0.874916672706604

Layer Weights Summary:

conv2d mean weight: 0.01015881 std: 0.12632039

conv2d_1 mean weight: 0.0008336163 std: 0.07186036

dense mean weight: 0.00024640534 std: 0.04437527

dense_1 mean weight: -0.014571281 std: 0.1368569

469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 4ms/step - accuracy: 0.8707 - loss: 0.3624 - val_accuracy: 0.8755 - val_loss: 0.3470

Epoch 3/10

468/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step - accuracy: 0.8897 - loss: 0.3066

Epoch: 3

Loss: 0.30485934019088745

Accuracy: 0.8899499773979187

Layer Weights Summary:

conv2d mean weight: 0.003983593 std: 0.13681383

conv2d_1 mean weight: -0.0014550275 std: 0.07882478

dense mean weight: -4.56706e-05 std: 0.04746696

dense_1 mean weight: -0.01622477 std: 0.1417586

469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 4ms/step - accuracy: 0.8897 - loss: 0.3066 - val_accuracy: 0.8878 - val_loss: 0.3163

Epoch 4/10

462/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step - accuracy: 0.8989 - loss: 0.2750

Epoch: 4

Loss: 0.27622711658477783

Accuracy: 0.8992999792098999

Layer Weights Summary:

conv2d mean weight: -9.399942e-06 std: 0.1455042

conv2d_1 mean weight: -0.003833485 std: 0.08469939

dense mean weight: -0.00037753157 std: 0.050493307

dense_1 mean weight: -0.017900404 std: 0.14670017

469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 4ms/step - accuracy: 0.8989 - loss: 0.2750 - val_accuracy: 0.8950 - val_loss: 0.2961

Epoch 5/10

467/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step - accuracy: 0.9086 - loss: 0.2562

Epoch: 5

Loss: 0.25329428911209106

Accuracy: 0.9078999757766724

Layer Weights Summary:

conv2d mean weight: -0.0022411658 std: 0.15345915

conv2d_1 mean weight: -0.005370209 std: 0.09046954

dense mean weight: -0.00056368805 std: 0.053481117

dense_1 mean weight: -0.019490158 std: 0.15220065

469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 4ms/step - accuracy: 0.9086 - loss: 0.2561 - val_accuracy: 0.8990 - val_loss: 0.2787

Epoch 6/10

460/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.9162 - loss: 0.2298

Epoch: 6

Loss: 0.23247437179088593

Accuracy: 0.9148499965667725

Layer Weights Summary:

conv2d mean weight: -0.004151346 std: 0.16122614

conv2d_1 mean weight: -0.0071428516 std: 0.095613144

dense mean weight: -0.00080812315 std: 0.056436583

dense_1 mean weight: -0.021087524 std: 0.15753472

469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 5ms/step - accuracy: 0.9162 - loss: 0.2298 - val_accuracy: 0.9054 - val_loss: 0.2687

Epoch 7/10

459/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step - accuracy: 0.9211 - loss: 0.2130

Epoch: 7

Loss: 0.2153884768486023

Accuracy: 0.920283317565918

Layer Weights Summary:

conv2d mean weight: -0.006444933 std: 0.16873476

conv2d_1 mean weight: -0.008472819 std: 0.1005313

dense mean weight: -0.0009400007 std: 0.059496317

dense_1 mean weight: -0.022920564 std: 0.16350453

469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 4ms/step - accuracy: 0.9211 - loss: 0.2131 - val_accuracy: 0.9077 - val_loss: 0.2570

Epoch 8/10

466/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step - accuracy: 0.9292 - loss: 0.1951

Epoch: 8

Loss: 0.2018720507621765

Accuracy: 0.9258666634559631

Layer Weights Summary:

conv2d mean weight: -0.007906069 std: 0.17501335

conv2d_1 mean weight: -0.009974346 std: 0.10516311

dense mean weight: -0.0012264185 std: 0.06287448

dense_1 mean weight: -0.024977598 std: 0.16947323

469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 4ms/step - accuracy: 0.9292 - loss: 0.1952 - val_accuracy: 0.9068 - val_loss: 0.2541

Epoch 9/10

465/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step - accuracy: 0.9327 - loss: 0.1841

Epoch: 9

Loss: 0.18519580364227295

Accuracy: 0.932283341884613

Layer Weights Summary:

conv2d mean weight: -0.008588804 std: 0.18178742

conv2d_1 mean weight: -0.011458854 std: 0.10974312

dense mean weight: -0.001418136 std: 0.06599558

dense_1 mean weight: -0.026930463 std: 0.17582108

469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 4ms/step - accuracy: 0.9327 - loss: 0.1841 - val_accuracy: 0.9072 - val_loss: 0.2617

Epoch 10/10

464/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step - accuracy: 0.9366 - loss: 0.1689

Epoch: 10

Loss: 0.1722087413072586

Accuracy: 0.9358833432197571

Layer Weights Summary:

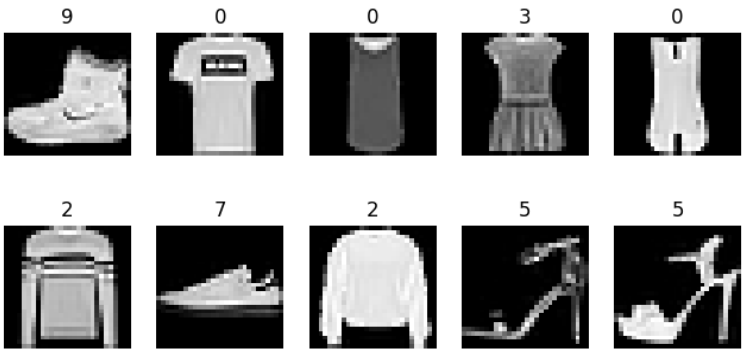
conv2d mean weight: -0.009597924 std: 0.18916866

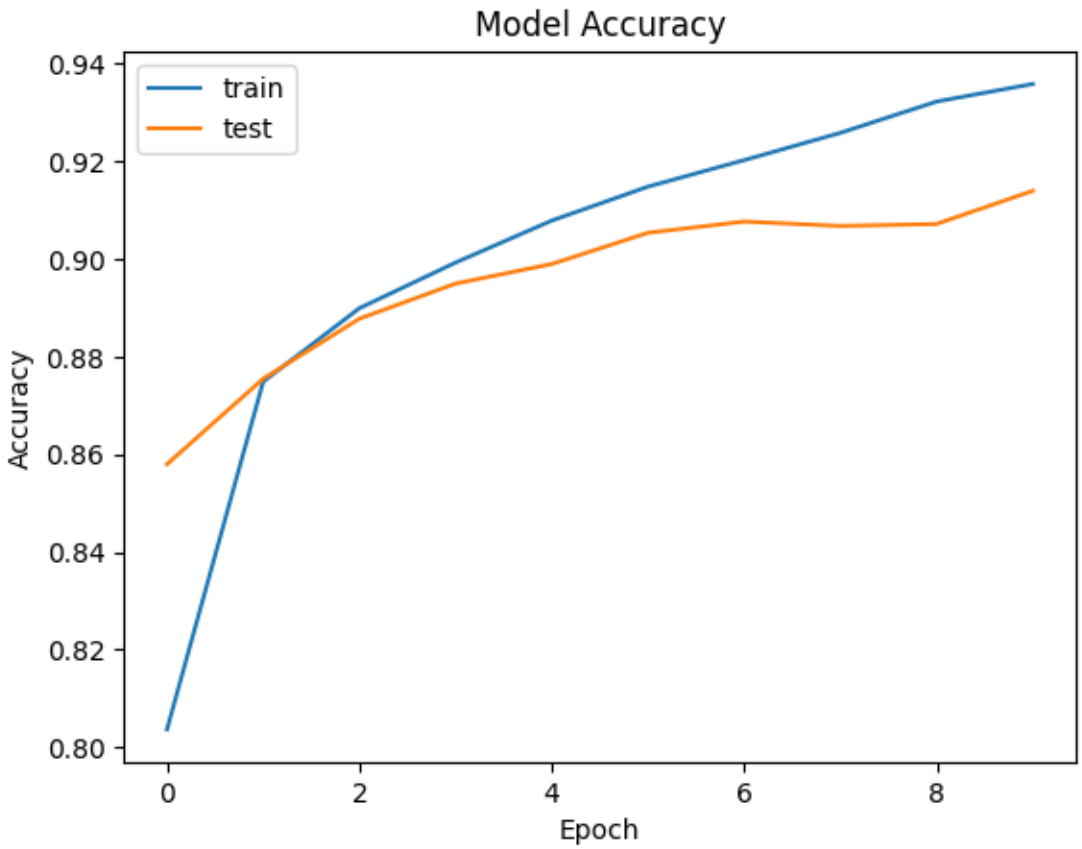
conv2d_1 mean weight: -0.011559088 std: 0.11419786

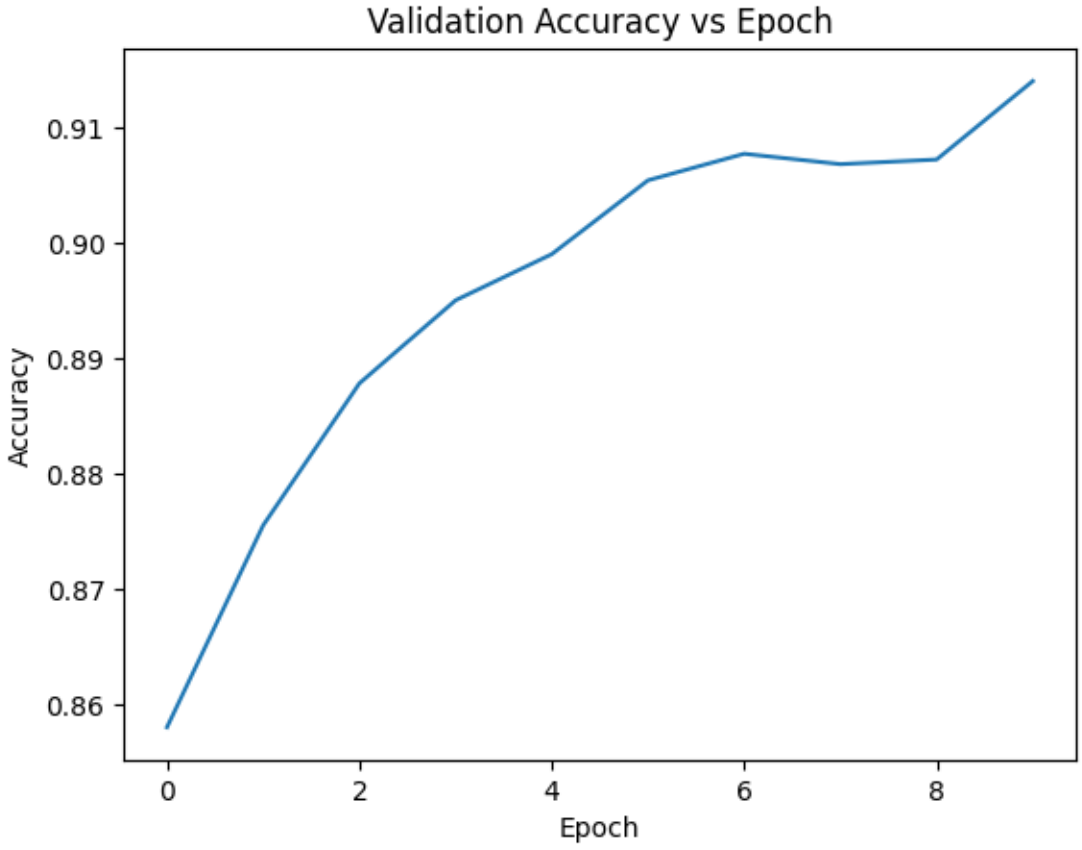
dense mean weight: -0.0013228639 std: 0.06923403

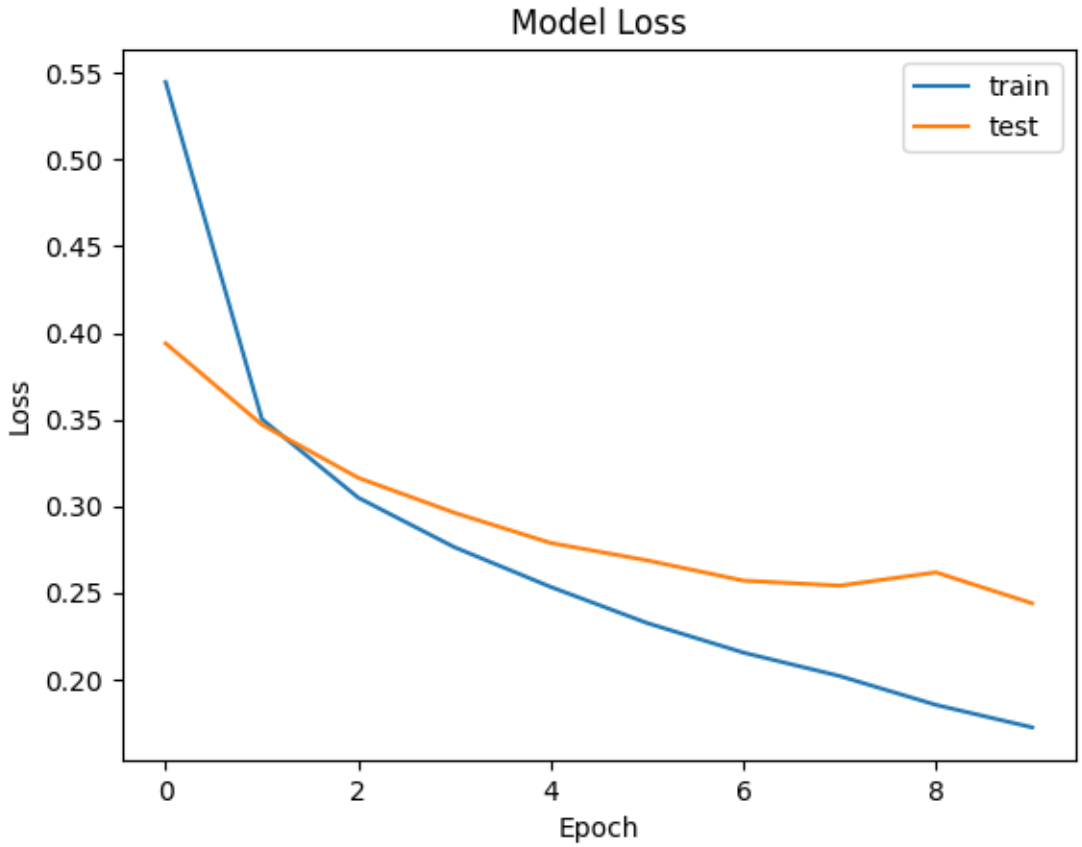
dense_1 mean weight: -0.02891459 std: 0.18263973

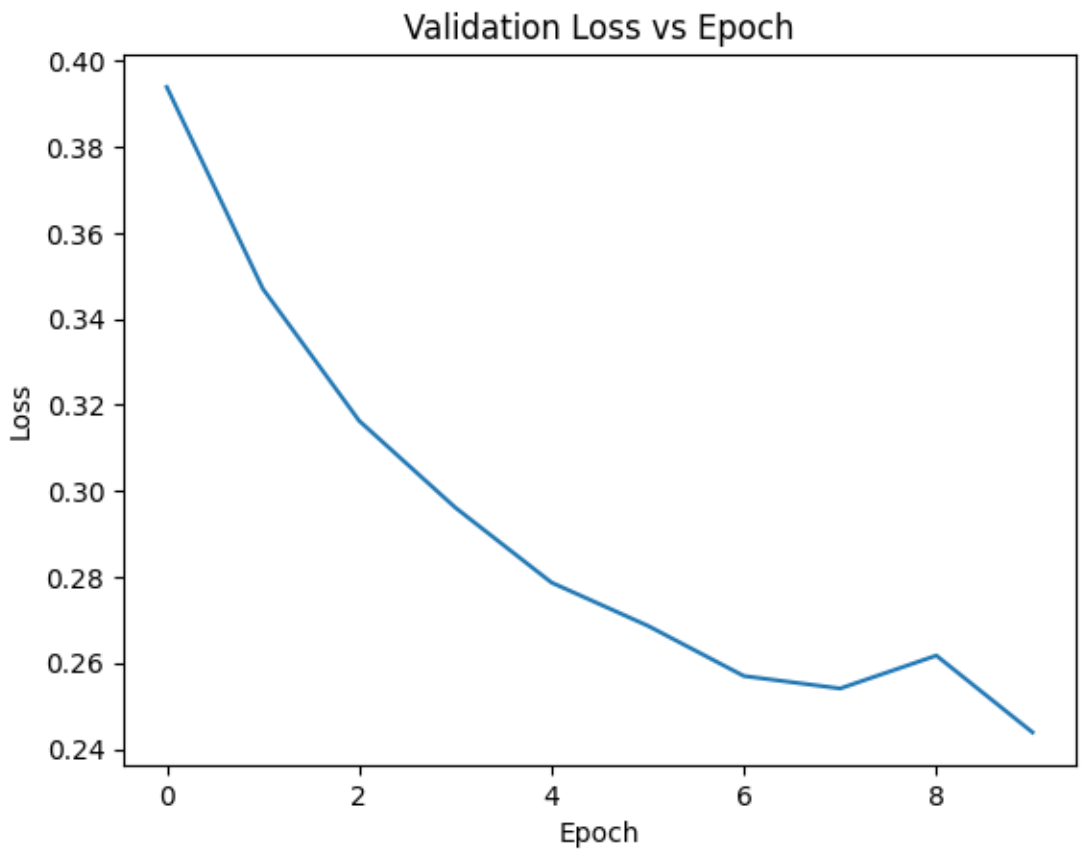
469/469 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 4ms/step - accuracy: 0.9366 - loss: 0.1689 - val_accuracy: 0.9140 - val_loss: 0.2438

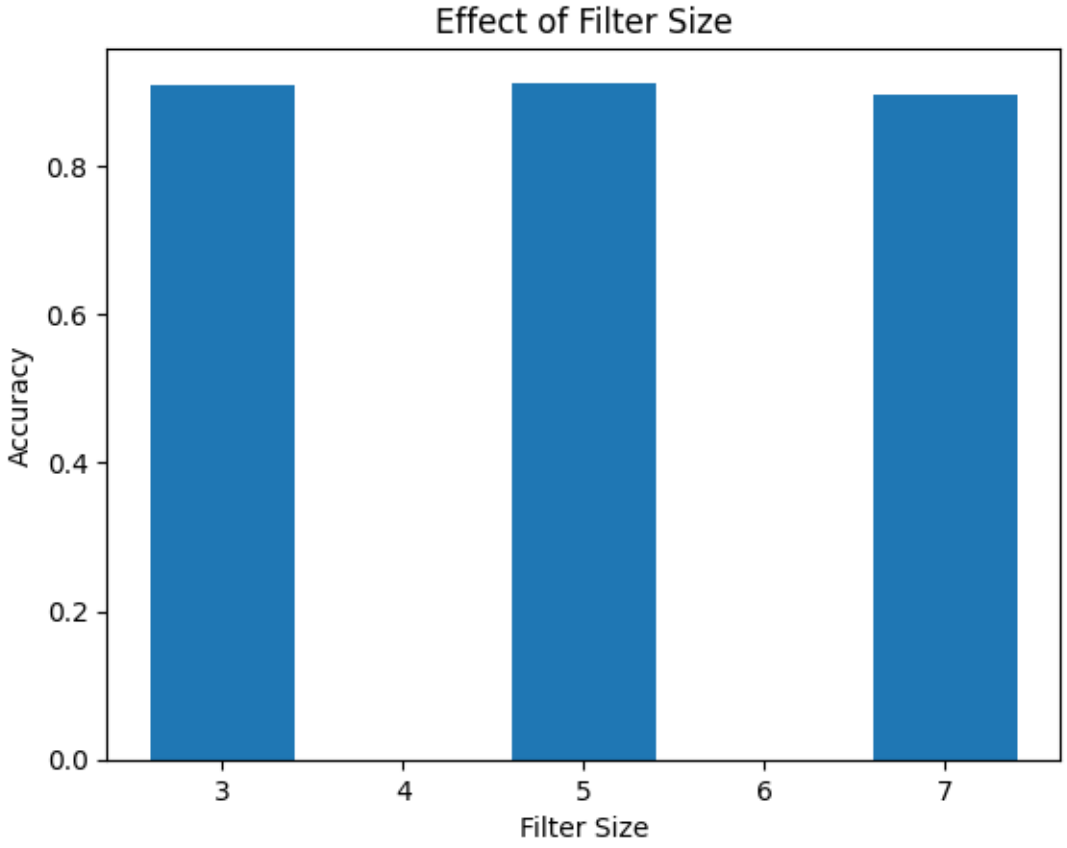


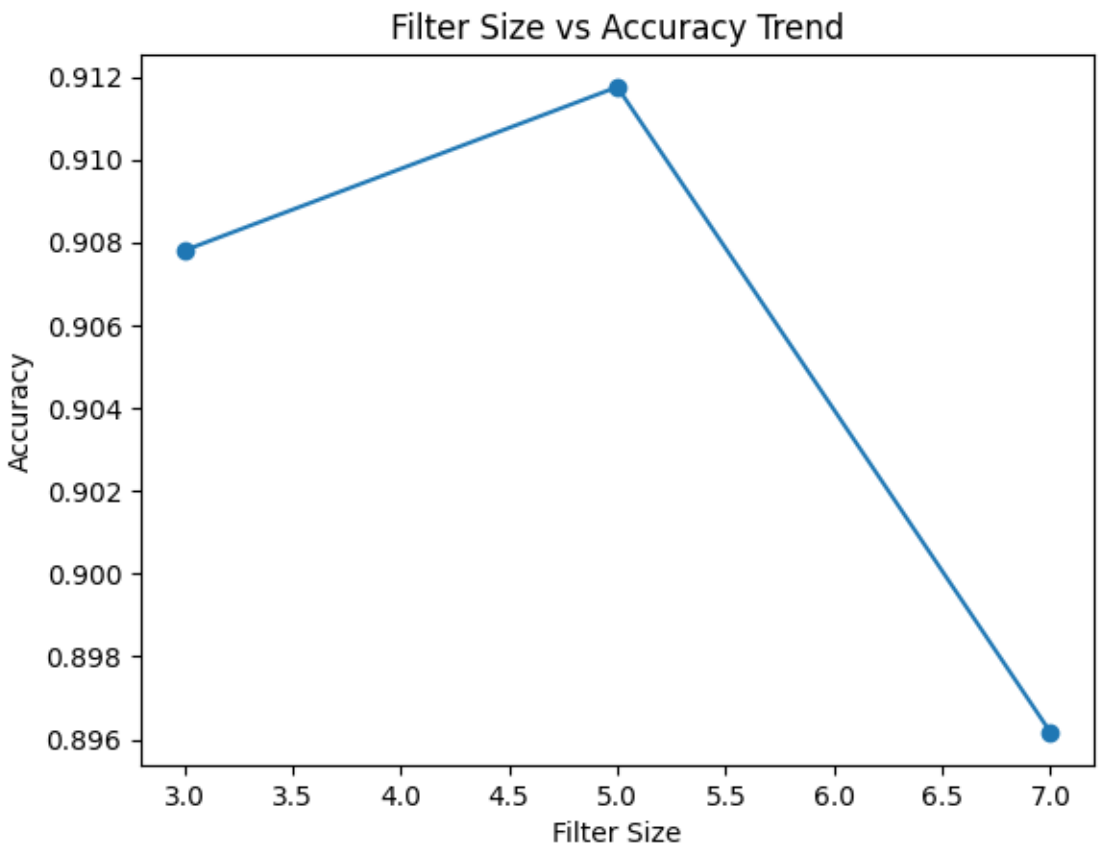

Model Accuracy
train
test
Accuracy
Epoch
0.94
0.92
0.90
0.88
0.86
0.84
0.82
0.80
0
2
4
6
8

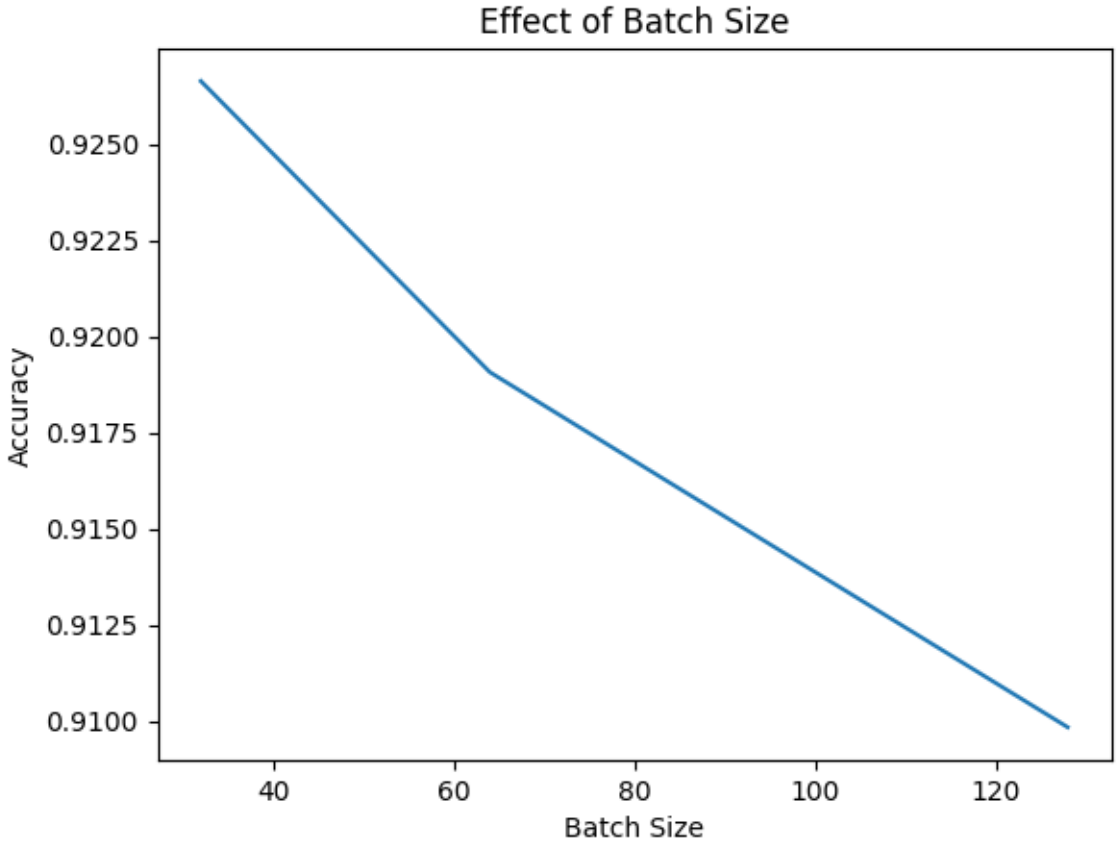

Validation Accuracy vs Epoch
Accuracy
Epoch
0.91
0.90
0.89
0.88
0.87
0.86
0
2
4
6
8

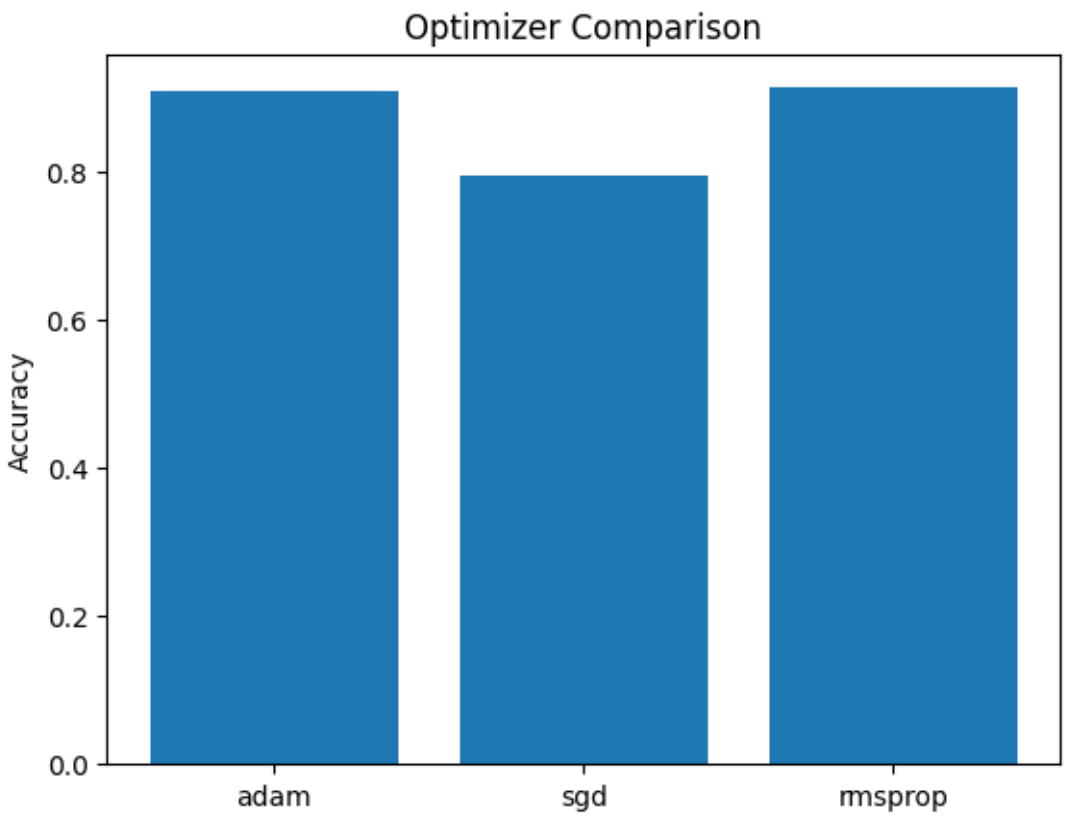

Model Loss
train
test
0.55
0.50
0.45
0.40
0.35
0.30
0.25
0.20
Loss
0
2
4
6
8
Epoch

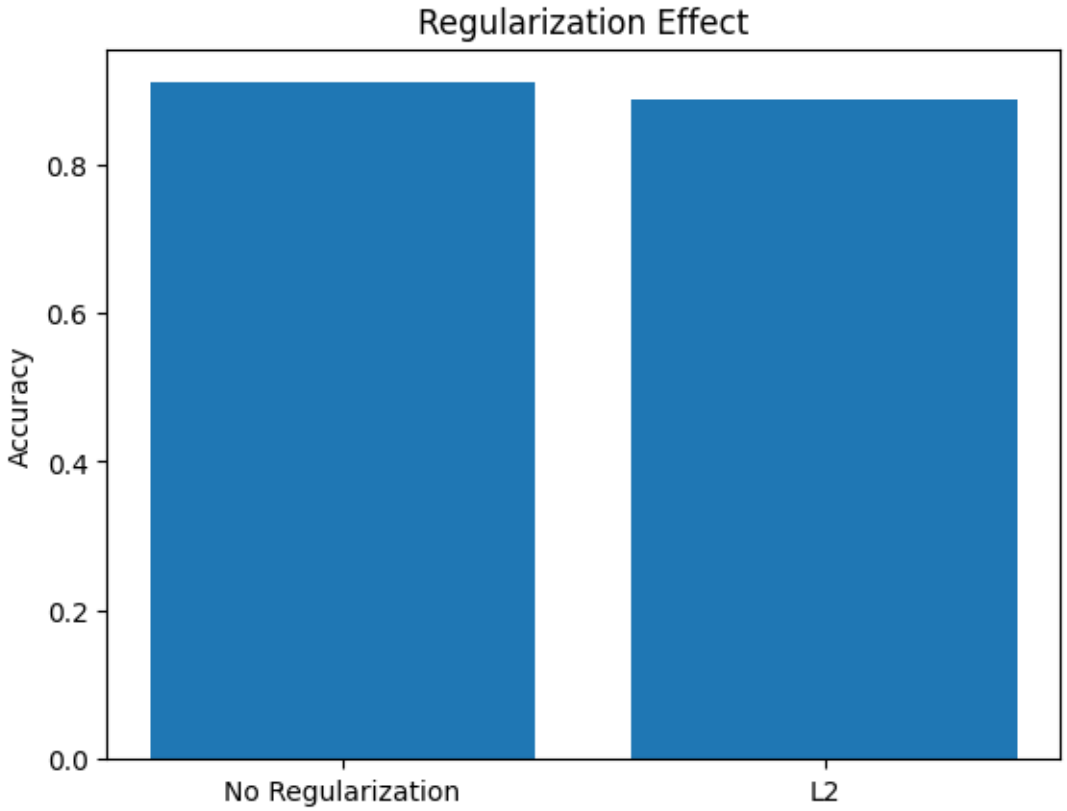

Validation Loss vs Epoch
Loss
Epoch
0.40
0.38
0.36
0.34
0.32
0.30
0.28
0.26
0.24
0
2
4
6
8


Effect of Filter Size
Accuracy
Filter Size
0.8
0.6
0.4
0.2
0.0
3
4
5
6
7


Filter Size vs Accuracy Trend
0.912
0.910
0.908
0.906
0.904
0.902
0.900
0.898
0.896
Accuracy
3.0
3.5
4.0
4.5
5.0
5.5
6.0
6.5
7.0
Filter Size


Effect of Batch Size
0.9250
0.9225
0.9200
0.9175
0.9150
0.9125
0.9100
Accuracy
40
60
80
100
120
Batch Size


Optimizer Comparison
Accuracy
0.0
0.2
0.4
0.6
0.8
adam
sgd
rmsprop


Regularization Effect
Accuracy
0.0
0.2
0.4
0.6
0.8
No Regularization
L2

Description of Code :-

loading required libraries:-
The program begins by importing the necessary libraries required for implementing the convolutional neural network. TensorFlow and Keras are used for building and training the deep learning model, while NumPy is used for numerical computations. Matplotlib is used to visualize the dataset and plot graphs for analyzing model performance.

- tensorflow and keras are used to build and train the convolutional neural network
- numpy is used for handling numerical arrays and performing mathematical operations
- matplotlib is used for visualizing images and plotting graphs for analysis

loading the fashion mnist dataset:-
The fashion mnist dataset is loaded using the keras datasets module. The dataset contains images of fashion products and their corresponding labels. The dataset is automatically divided into training data and testing data, which helps in training the model and evaluating its performance.

- the dataset contains grayscale images of size 28 × 28 pixels
- the training dataset contains 60000 images used for learning
- the testing dataset contains 10000 images used for evaluation
- each image belongs to one of ten clothing categories

data preprocessing and normalization:-
Before training the model, the input data is preprocessed. The pixel values of images range from 0 to 255, so they are normalized to a range between 0 and 1 by dividing by 255. This improves numerical stability and helps the model converge faster during training. The images are also reshaped to include a channel dimension required for convolutional neural networks.

- pixel values are normalized to improve training stability
- normalization helps faster convergence of the model
- images are reshaped to include a channel dimension for cnn input

visualizing dataset sample images:-
A few sample images from the training dataset are displayed using matplotlib. This step helps verify that the dataset has been loaded correctly and also gives a visual understanding of the fashion items present in the dataset.

- sample images are displayed using matplotlib plotting functions
- each image is shown along with its corresponding label
- visualization helps understand the dataset before training the model

model architecture creation:-
A function is created to define the convolutional neural network architecture. The model consists of convolution layers, pooling layers, flatten layers, and dense layers. The convolution layers extract spatial features from the images, while the dense layers perform the classification.

- convolution layers extract features such as edges and textures
- pooling layers reduce spatial dimensions and computational complexity
- flatten layer converts feature maps into a one dimensional vector
- dense layers perform the final classification task

model compilation:-
After defining the model architecture, the model is compiled. During compilation, the loss function, optimizer, and evaluation metrics are specified. The categorical cross entropy loss function is used for classification tasks and the adam optimizer is used to update the model weights efficiently.

- loss function measures the error between predicted and actual labels
- adam optimizer adjusts weights using adaptive learning rates
- accuracy metric is used to evaluate classification performance

training the cnn model:-
The model is trained using the training dataset for several epochs. During each epoch, the model processes the data in batches and updates the weights through the backpropagation algorithm. The training process gradually reduces the loss and improves accuracy.

- training is performed for multiple epochs
- batch processing improves computational efficiency
- backpropagation updates the weights to minimize loss
- model accuracy improves with each epoch

monitoring epoch level details:-
A custom callback is used to display detailed information after each epoch. This includes training loss, accuracy, and summary statistics of layer weights. Monitoring these values helps understand how the model learns during training.

- epoch number, loss, and accuracy are printed after each epoch
- mean and standard deviation of layer weights are displayed
- helps track how the model parameters change during training

model evaluation on test data:-
After training is completed, the model is evaluated using the testing dataset. This step measures how well the model performs on unseen data and provides an estimate of the model's generalization capability.

- testing dataset is used for performance evaluation
- test accuracy indicates how well the model classifies new images
- evaluation helps measure the effectiveness of the trained model

performance visualization using graphs:-
Several graphs are generated to analyze the training process and the effect of different hyperparameters. These graphs help in understanding how the model improves over time and how different configurations affect performance.

- training accuracy vs epoch graph shows learning progress
- training loss vs epoch graph shows reduction of error
- filter size vs accuracy graph analyzes convolution performance
- batch size vs accuracy graph studies training efficiency
- optimizer comparison graph evaluates optimization methods
- regularization comparison graph studies overfitting control

overall program workflow:-
The complete program follows a structured workflow that includes dataset loading, preprocessing, model construction, training, evaluation, and performance analysis. This workflow demonstrates

how convolutional neural networks can effectively learn image features and perform classification tasks with high accuracy.

• dataset is loaded and preprocessed for training
• cnn model architecture is created and compiled
• model is trained using backpropagation and optimization algorithms
• evaluation and graphical analysis help understand model performance

Summary of Performance Evaluation :-

The performance of the convolutional neural network model was evaluated using several metrics and graphical analyses during the training and testing phases. The model was trained on the fashion mnist dataset to classify different types of clothing images. During training, the accuracy increased steadily while the loss value decreased, indicating that the model was successfully learning meaningful patterns from the input data. The evaluation on the test dataset confirmed that the model was able to generalize well to unseen images.

different hyperparameters such as filter size, batch size, optimization algorithm, and regularization were varied to analyze their effect on the model performance. The results were observed through different graphs and accuracy measurements. These experiments helped in understanding how different configurations influence the learning capability and efficiency of the convolutional neural network.

• the training accuracy improved gradually with each epoch, showing that the model learned important image features over time

• the training loss decreased continuously, indicating that the prediction error reduced during training

• the validation accuracy remained close to the training accuracy, which suggests that the model was able to generalize well without significant overfitting

• smaller filter sizes such as 3 × 3 were more effective in capturing fine image features and produced better classification accuracy

• increasing the batch size improved computational efficiency but sometimes slightly affected the model's ability to generalize

• the adam optimizer achieved faster convergence and better performance compared to other optimizers such as sgd

• applying regularization helped control overfitting by preventing the model weights from becoming excessively large

• graphical analysis such as accuracy versus epoch and loss versus epoch clearly showed the learning behavior of the model

• comparison graphs for different hyperparameters helped identify the optimal configuration for achieving higher accuracy

overall, the convolutional neural network demonstrated strong performance in classifying fashion mnist images with high accuracy. The experiment also showed that proper selection of

hyperparameters and analysis through graphs can significantly improve the effectiveness and reliability of the deep learning model.

My Comments :-

From this experiment I learned the following :-

1. How convolutional neural networks can be used for image classification tasks. I understood how CNN models automatically extract important features such as edges, textures, and shapes from images without manually defining features.

2. I learned how to load and preprocess image datasets as this, the fashion mnist dataset using keras.

3. Got to know like how convolution layers, pooling layers, flatten layers, and dense layers work together in a CNN architecture to perform feature extraction and classification.

4. How activation functions, loss functions, and optimizers play an important role in training the model and updating the network weights using backpropagation.

5. I also understood different hyperparameters such as filter size, batch size, optimizer, and regularization can affect the performance and learning behavior of the model.